

Diffusion Models for Generating Images

Marios S. Pattichis, FEAMBES, Senior Member, National Academy of Inventors
Director of online M.Sc. In Computer Engineering in Applied Machine Learning And AI Systems Engineering
Department of Electrical and Computer Engineering
The University of New Mexico

References

The coding examples are adopted from:

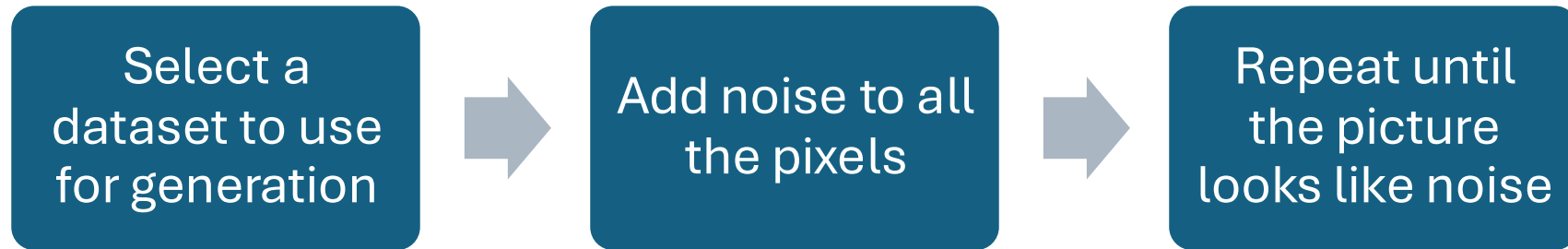
A. Géron (2025). *Hands-On Machine Learning with Scikit-Learn and PyTorch: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly.

The basic **learning models** are from:

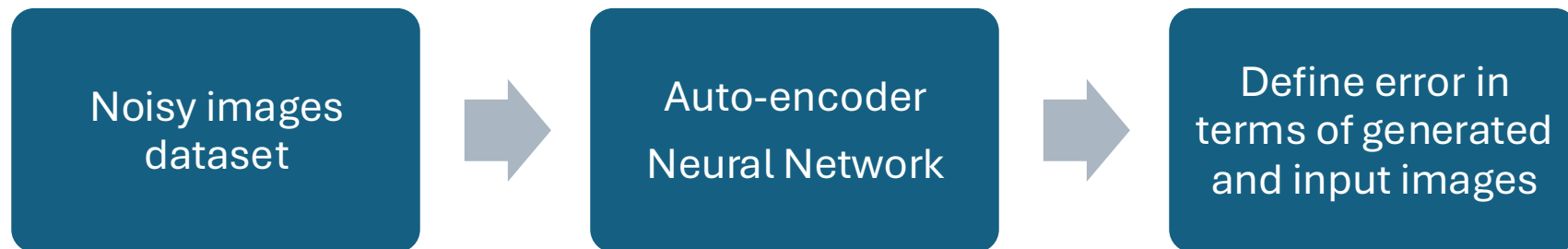
Torralba, A., Isola, P., & Freeman, W. T. (2024). *Foundations of Computer Vision*. The MIT Press. <https://visionbook.mit.edu/>.

The Diffusion Process: Training

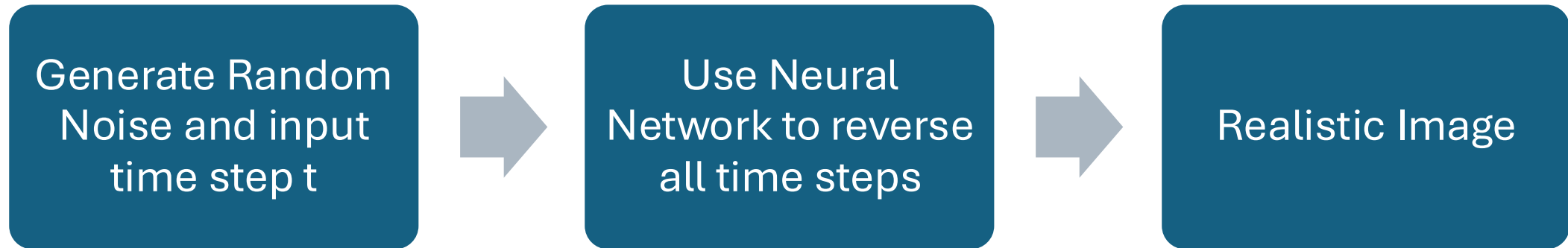
Step 1. Build a dataset to train the diffusion process:



Step 2. Train an auto-encoder neural network to remove noise from an image:



The Diffusion Process: Deployment



Training dataset: Create a sequence from any input image

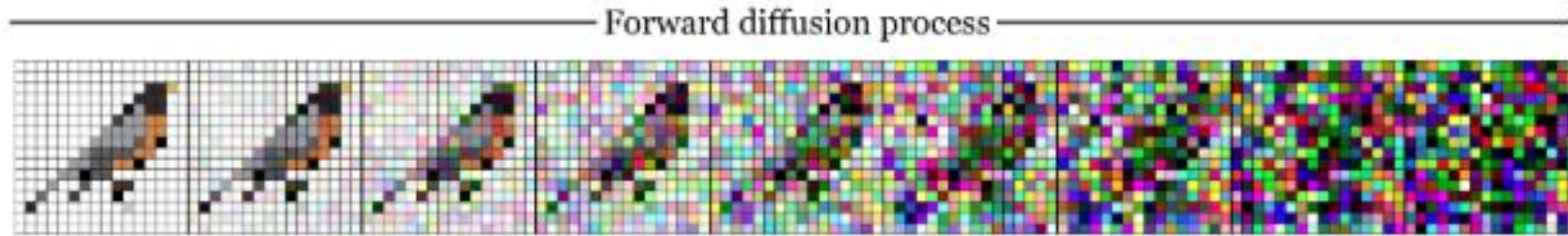
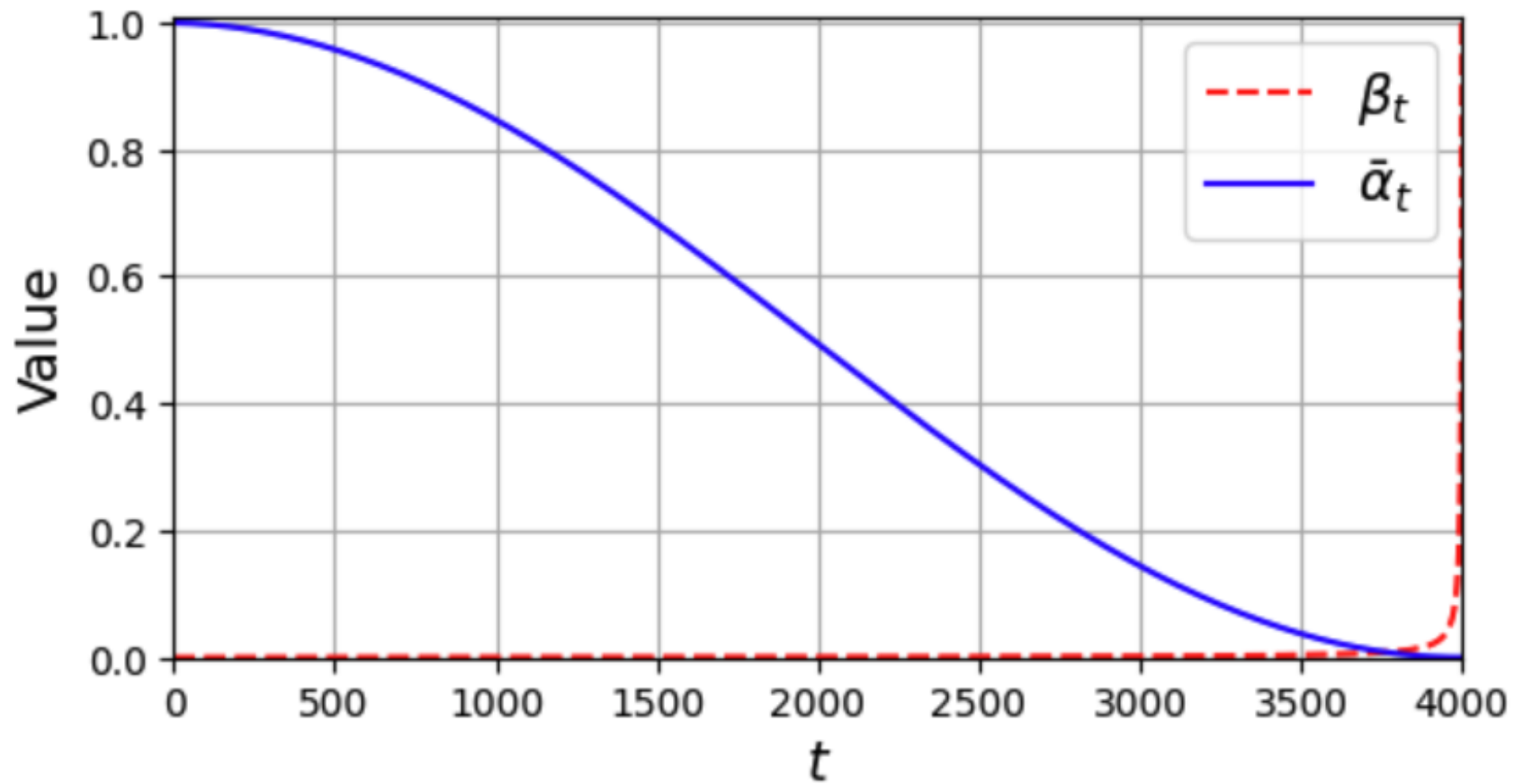


Figure 32.14: Forward diffusion process.

After T steps of this process, the image $\mathbf{x}_T$ looks like pure noise, if T is large enough. In fact, if we use the following noise process, then $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ as $T \rightarrow \infty$ (which follows from equation 4 in [4]).

$$\begin{aligned}\mathbf{x}_t &= \sqrt{(1 - \beta_t)}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_t \\ \epsilon_t &\sim \mathcal{N}(\mathbf{0}, \mathbf{I})\end{aligned}$$

Generate noisy images



```
1 def forward_diffusion(x0, t):
2     eps = torch.randn_like(x0) # this unscaled noise will be the target
3     xt = alpha_bars[t].sqrt() * x0 + (1 - alpha_bars[t]).sqrt() * eps
4     return xt, eps
```

Neural Net Goal: Reverse the sequence

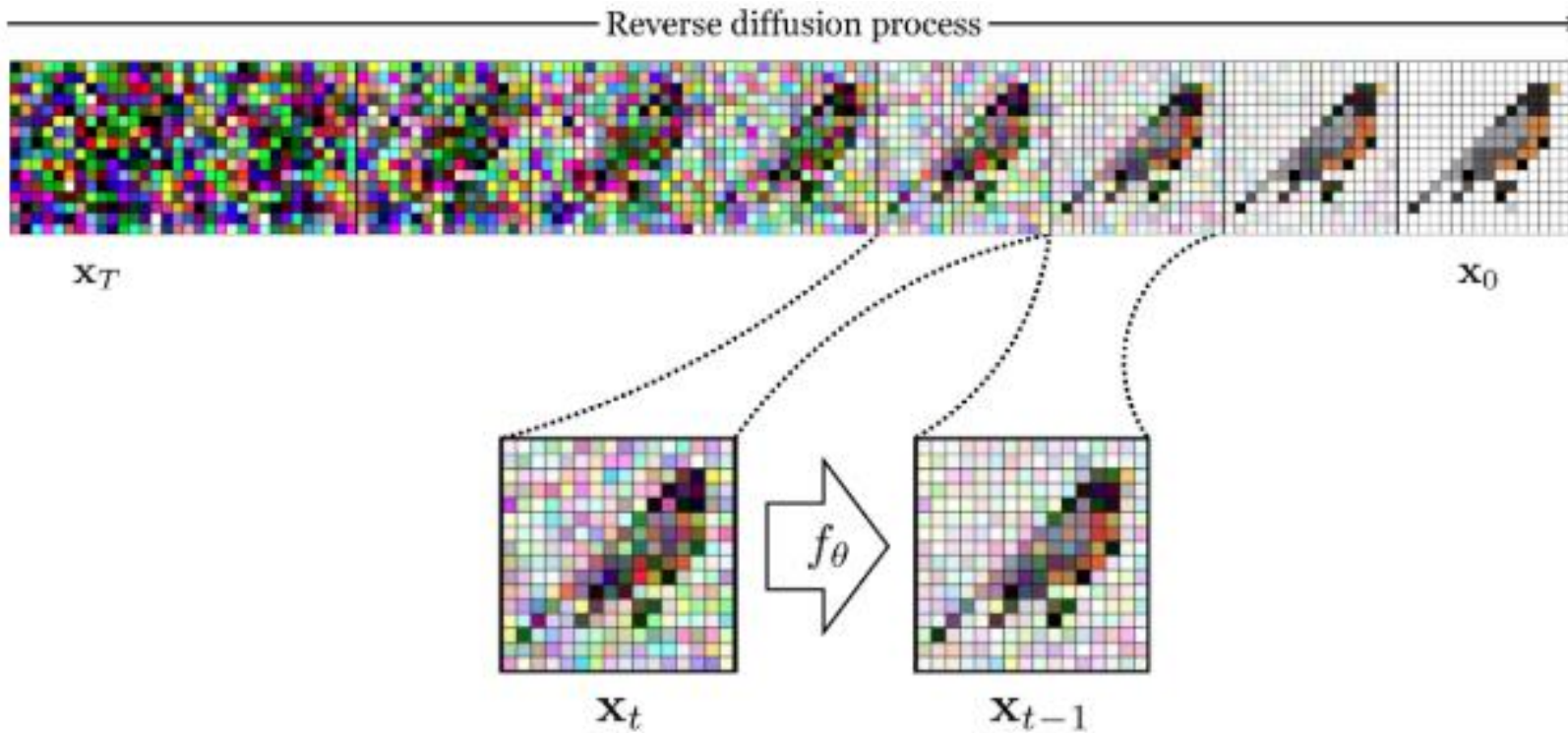


Figure 32.15: Reverse diffusion process.
The forward process creates supervision to train the reverse process.

$$\hat{\mathbf{x}}_{t-1} = f_\theta(\mathbf{x}_t, t)$$

Use an Auto-encoder as a Diffusion Model

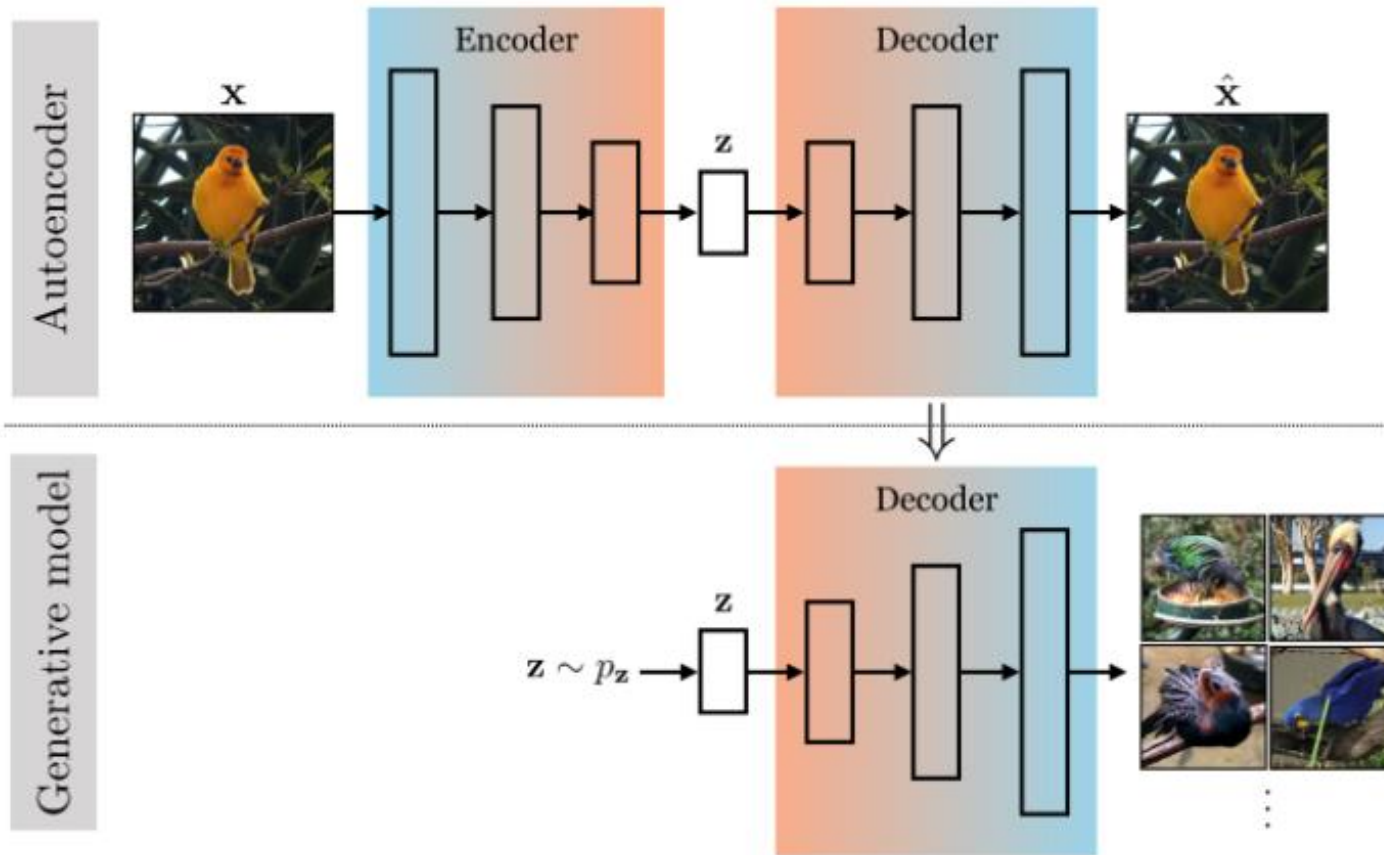
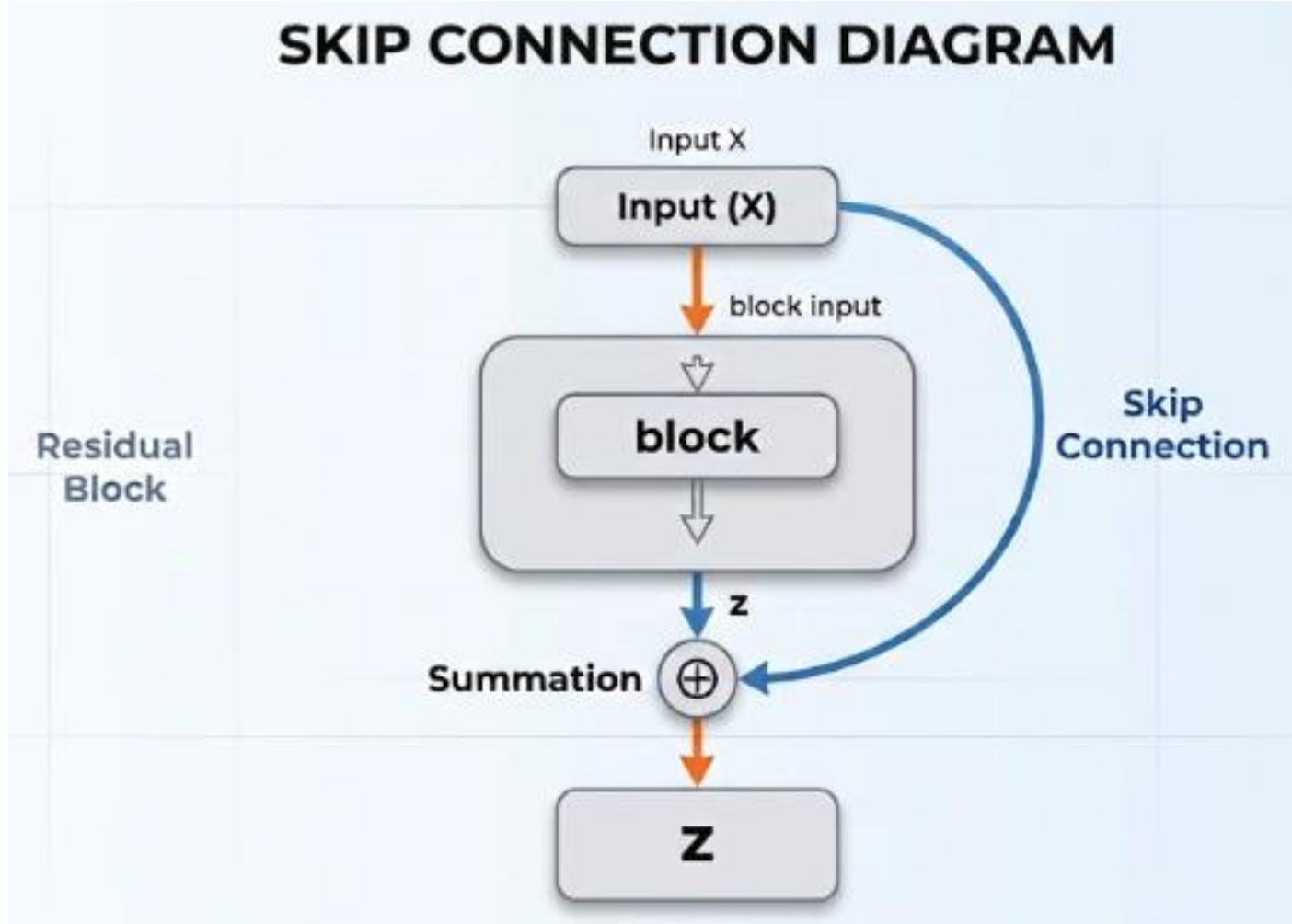


Figure 33.3: The relationship between an autoencoder and a generative model.

Adding a Skip Connection to the Neural Net



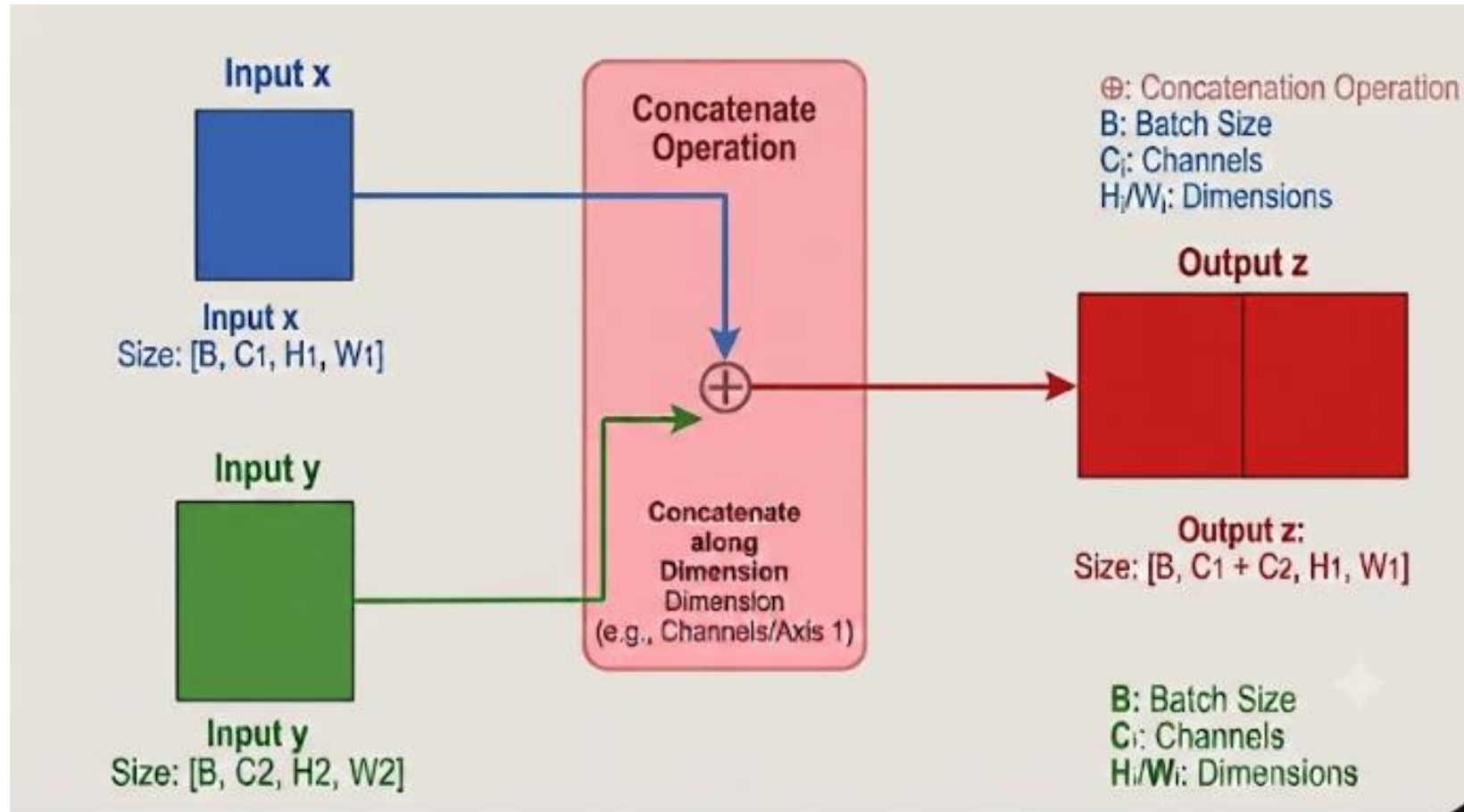
$$Z = z + x$$

Big advantage:

If the block produces zero output, the network propagates the input instead of propagating nothing.

The dimensions of x and z must be the same. Why?

Concatenation Operation



Encoder: Convert Images to a small vector

```
# Downsampling path
for block, skip_conv, time_adapter in zip(
    self.down_blocks, self.skip_convs, self.time_adapters_down):
    z = block(z)
    cross_skips.append(z)
    z = F.max_pool2d(z, 3, stride=2, padding=1)
    skip_link = skip_conv(skip)
    z = z + skip_link
    z = z + time_adapter(time_enc)[: , :, None, None]
    skip = z
```

Can you see the skip and append connections?

What do you expect to see after the pooling operation?

What information must be there?

Decoder: Convert small vector back to the image

```
# Upsampling path
for block, skip_up_conv, time_adapter in zip(
    self.up_blocks, self.skip_up_convs, self.time_adapters_up):
    z = block(z)
    z = F.interpolate(z, scale_factor=2, mode="nearest")
    skip_link = F.interpolate(skip, scale_factor=2, mode="nearest")
    skip_link = skip_up_conv(skip_link)
    z = z + skip_link
    z = z + time_adapter(time_enc)[: , :, None, None]
    cross_skip = cross_skips.pop()
    z = torch.cat([z, cross_skip], dim=1)
    skip = z
```

Note: Interpolation does the opposite of max pooling. It increases the dimensionality.

What are the issues with using interpolation for increasing dimensionality?

Image generator of a batch of images

```
def generate_ddim(model, batch_size=32, num_steps=50, eta=0.85):
    model.eval()
    with torch.no_grad():
        xt = torch.randn([batch_size, 1, 28, 28], device=device)
        times = torch.linspace(T - 1, 0, steps=num_steps + 1).long().tolist()
        for t, t_prev in zip(times[:-1], times[1:]):
            print(f"\rt = {t}", end=" ")
            t_batch = torch.full((batch_size, 1), t, device=device)
            sample = DiffusionSample(xt, t_batch)
            eps_pred = model(sample)
            x0 = ((xt - (1 - alpha_bars[t]).sqrt() * eps_pred)
                 / (alpha_bars[t].sqrt()))
            abar_t_prev = alpha_bars[t_prev]
            variance = eta * (1 - abar_t_prev) / (1 - alpha_bars[t]) * betas[t]
            sigma_t = variance.sqrt()
            pred_dir = (1 - abar_t_prev - sigma_t**2).sqrt() * eps_pred
            noise = torch.randn_like(xt)
            xt = abar_t_prev.sqrt() * x0 + pred_dir + sigma_t * noise

        return torch.clamp((xt + 1) / 2, 0, 1) # from [-1, 1] range to [0, 1]
```

```
torch.manual_seed(42) # extra code – ensures reproducibility
X_gen_ddim = generate_ddim(diffusion_model, num_steps=500)
```

Discuss the code segments:

Generate random noise:

`xt = torch.randn(...)`

Initialize times backwards:

`times = torch.linspace(...)`

Apply a countdown for-loop.

Generate images at different time steps using `xt`.

Predict the noise using:

`eps_pred = model(sample)`

Remove the noise from the image.

Call the function to generate an image

Example Output



Do the images
appear realistic?